# Лекція №3 (2 години). Блокова модель CSS та позиціювання елементів.

## План лекції

1. Блокова модель CSS (CSS Box Model)
2. Типи відображення елементів (Властивість `display`)
3. Позиціювання елементів (Властивість `position`)
4. Контекст накладання (z-index)
5. Плаваючі елементи (Floats)

## Перелік умовних скорочень

- **CSS** (Cascading Style Sheets) — каскадні таблиці стилів.
- **HTML** (HyperText Markup Language) — мова гіпертекстової розмітки.
- **W3C** (World Wide Web Consortium) — консорціум Всесвітнього павутиння (організація, що розробляє стандарти інтернету).
- **UI** (User Interface) — інтерфейс користувача.
- **UX** (User Experience) — користувацький досвід.

## Вступ

Сучасна веб-розробка неможлива без глибокого розуміння того, як браузер відображає елементи на сторінці. Основою цього процесу є **блокова модель CSS** (Box Model). Усе, що ми бачимо на веб-сторінці — це прямокутники, які вкладаються один в одного, розташовуються поруч або перекриваються. Розуміння цієї моделі дозволяє розробникам точно контролювати розміри, відступи та межі елементів.

Другим критично важливим аспектом є **позиціювання**. Не завжди достатньо стандартного потоку документа (коли елементи йдуть один за одним). Часто виникає потреба закріпити меню навігації на екрані, створити спливаюче вікно, яке перекриває контент, або точно розмістити іконку всередині кнопки. Для вирішення цих задач існують спеціальні властивості позиціювання та управління шарами (z-index).

Мета цієї лекції — розібрати анатомію HTML-елемента з точки зору CSS і навчитися ефективно керувати його розміщенням на сторінці.

# 1. Блокова модель CSS (CSS Box Model)

Кожен елемент на веб-сторінці браузер розглядає як прямокутний блок. Блокова модель CSS описує структуру цього блоку та визначає, як обчислюються його кінцеві розміри.

Блок складається з чотирьох основних областей (від центру до країв):

1. **Content (Контент)** — вміст елемента: текст, зображення, відео тощо. Розміри цієї області визначаються властивостями `width` та `height`.
2. **Padding (Внутрішні відступи)** — прозорий простір між контентом і межею (border) елемента. Фон елемента поширюється на цю область.
3. **Border (Межа)** — лінія, яка оточує контент і внутрішні відступи (padding). Може мати колір, товщину та стиль (суцільна, пунктирна тощо).
4. **Margin (Зовнішні відступи)** — прозорий простір поза межею елемента. Відокремлює елемент від сусідніх елементів на сторінці.

**Приклад структури блокової моделі:**

```
.box {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 15px;
}
```

## Проблема класичної моделі (content-box)

За замовчуванням у CSS застосовується властивість `box-sizing: content-box`. Це означає, що властивості `width` та `height` задають розміри **лише контентної області**. У прикладі вище реальна ширина елемента на екрані складатиме: `200px (width) + 20px (padding-left) + 20px (padding-right) + 5px (border-left) + 5px (border-right) = 250px`. Це було незручно, оскільки додавання відступів "ламало" сітку макету.

## Рішення: border-box

Сучасний підхід полягає у використанні `box-sizing: border-box`. В цьому режимі `width` та `height` визначають **повний розмір елемента** (включаючи контент, padding та border). Якщо задати `width: 200px`, контентна область автоматично стиснеться так, щоб у ці 200 пікселів влізли і border, і padding.

**Best Practice (Найкраща практика)** — застосовувати скидання `box-sizing` для всіх елементів на сторінці:

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

### Схлопування зовнішніх відступів (Margin Collapsing)

Це специфічна поведінка браузерів: коли вертикальні `margin` двох блоків (один над одним) зустрічаються, вони не сумуються. Замість цього береться **найбільше значення** з двох. *Приклад:* Блок 1 має `margin-bottom: 20px`, Блок 2 (під ним) має `margin-top: 30px`. Відстань між ними буде `30px`, а не `50px`. Схлопування працює лише для вертикальних відступів блокових елементів у нормальному потоці.

## 2. Типи відображення елементів (Властивість `display`)

Поведінка елемента в потоці документа залежить від його типу відображення.

- `display: block` **(Блокові елементи)**
  - Займають всю доступну ширину батьківського контейнера (100% ширини).
  - Завжди починаються з нового рядка.
  - Можна задавати `width`, `height`, `margin`, `padding`.
  - Приклади: `<div>`, `<p>`, `<h1>` ... `<h6>`, `<ul>`, `<li>`.
- `display: inline` **(Рядкові елементи)**
  - Займають ширину, необхідну лише для їх контенту.
  - Не починаються з нового рядка (шикуються в лінію).
  - **Не можна** задати ширину (`width`) або висоту (`height`).
  - Вертикальні `margin` та `padding` не впливають на розташування сусідніх рядків тексту.
  - Приклади: `<span>`, `<a>`, `<strong>`, `<em>`.
- `display: inline-block` **(Рядково-блокові елементи)**
  - Поєднують риси обох типів: розташовуються в рядок (як inline), але їм **можна** задавати `width`, `height`, та повноцінні відступи (як block).

- Приклади застосування: кнопки, картки товарів (історично, до появи Flexbox).
- Стандартні елементи з такою поведінкою: `<img>`, `<input>`, `<button>`.

- **Приховування елементів**
  - `display: none;` — елемент повністю видаляється візуально і вилучається з потоку документа. Інші елементи займають його місце.
  - `visibility: hidden;` — елемент стає прозорим/невидимим, але продовжує займати своє місце в макеті (фізично присутній для потоку).

## 3. Позиціювання елементів (Властивість `position`)

За замовчуванням браузер розташовує елементи один за одним: блокові зверху вниз, рядкові — зліва направо. Це називається **нормальним потоком документа** (Normal flow). Властивість `position` дозволяє виймати елементи з цього потоку або зміщувати їх.

Керування позицією відбувається за допомогою координатних властивостей: `top`, `right`, `bottom`, `left`.

- **`position: static` (Статичне — за замовчуванням)**
  - Елемент знаходиться у нормальному потоці.
  - Властивості `top`, `left` тощо на нього **не діють**.
  - `z-index` не працює.
- **`position: relative` (Відносне)**
  - Елемент залишається у нормальному потоці (його початкове місце зберігається).
  - Властивості `top`, `left` зміщують елемент **відносно його початкового положення**.
  - Елемент може перекривати інші елементи.
  - *Важлива особливість:* часто використовується як координатна система ("якір") для абсолютно спозиційованих нащадків.
- **`position: absolute` (Абсолютне)**
  - Елемент **повністю випадає з нормального потоку** (його місце займають інші елементи, ніби його ніколи не було).
  - Позиціюється (`top`, `left`...) відносно **найближчого батьківського елемента, який має позиціювання** відмінне від `static` (зазвичай батьку задають `position: relative`).

- Якщо такого батька немає, позиціюється відносно всього документа (вікна браузера `<html>`).
- *Приклад:* крестик "закрити" в правому верхньому куті модального вікна.

```
.card {
  /* Батько */
  position: relative;
  width: 300px;
  height: 200px;
}
.badge {
  /* Дочірній */
  position: absolute;
  top: -10px;
  right: -10px;
}
```

- **`position: fixed` (Фіксоване)**
  - Елемент випадає з потоку документа.
  - Завжди позиціюється **відносно вікна браузера** (viewport), незалежно від батьків.
  - Елемент залишається "прибитим" до одного місця навіть під час прокручування сторінки.
  - *Приклад:* плаваюче навігаційне меню (header), кнопка "Догори".
- **`position: sticky` (Липке)**
  - Гібрид `relative` та `fixed`.
  - Елемент поводиться як `relative` (прокручується разом з контентом), поки сторінка не доскролиться до певного порогу (наприклад, `top: 0`). У цей момент він стає `fixed` і "прилипає".
  - Коли батьківський контейнер закінчується, липкий елемент "відлипає" і рухається вгору разом із батьком.

## 4. Контекст накладання (z-index)

Коли елементи перекривають один одного (наприклад, через абсолютне позиціювання або від'ємні марджини), браузер повинен вирішити, що відобразити зверху. За замовчуванням, елемент, який написаний у HTML-коді нижче, буде перекривати елементи, які написані вище.

Властивість `z-index` дозволяє змінити цей порядок. Це працює як вісь Z (глибина), спрямована на користувача.

- Працює **тільки** для спозиційованих елементів ( `relative` , `absolute` , `fixed` , `sticky` ).
- Значення можуть бути додатними, нульовими або від'ємними (наприклад, `z-index: 10;` , `z-index: -1;` ).
- Елемент із більшим `z-index` перекриває елемент із меншим.

**Важливо: Контексти накладання (Stacking Contexts)** `z-index` працює не глобально по всій сторінці, а в межах локальних контекстів. Якщо Батько А має `z-index: 1` , а Батько Б має `z-index: 2` , то будь-який дочірній елемент Батька А (навіть із `z-index: 9999` ) ніколи не зможе перекрити Батька Б.

## 5. Плаваючі елементи (Floats)

Властивість `float` була створена для "обтікання" зображень текстом, подібно до того, як це робиться у традиційній типографії та газетних статтях.

- `float: left;` — елемент притискається до лівого краю свого контейнера, а текст і вбудовані елементи обтікають його праворуч.
- `float: right;` — елемент притискається до правого краю, а текст обтікає його ліворуч.
- `float: none;` — (за замовчуванням) елемент не плаває.

**Приклад обтікання зображення текстом:**

```
<img src="logo.png" style="float: left; margin-right: 15px;" />
<p>
  Цей текст буде обтікати зображення з правого боку. Якщо тексту багато, він
  продовжиться під зображенням, коли пройде його висоту.
</p>
```

### Проблема "випадіння" з контейнера

Основна проблема з плаваючими елементами полягає в тому, що вони частково вилучаються з нормального потоку документа. Якщо батьківський контейнер містить **лише** плаваючі елементи ( `float: left` або `float: right` ), його висота "склопнеться" до нуля (стане `0px` ), оскільки браузер не враховує висоту плаваючих нащадків. Це призводить до того, що фон батька зникає, а наступні неплаваючі елементи наїжджають на плаваючі.

### Вирішення проблеми: Властивість `clear` та хак `.clearfix`

Щоб сказати браузеру "зупини обтікання і повернися до нормального потоку", використовується властивість `clear` (значення `left`, `right`, або `both`). Вона забороняє елементу підніматися поруч із плаваючим сусідом.

**Історичний хак `.clearfix`:** Оскільки створення додаткового порожнього `<div style="clear: both;"></div>` після плаваючих елементів засмічувало HTML-код, розробники придумали CSS-рішення, яке застосовується до самого **батьківського контейнера**:

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

*Історична довідка:* Протягом багатьох років (до появи Flexbox та Grid) розробники будували цілі сітки сайтів (колонки) за допомогою `float`. Сьогодні використання `float` для макетування вважається застарілим і поганою практикою (anti-pattern). Зараз цю властивість слід застосовувати виключно за прямим призначенням — для обтікання контенту навколо зображень чи окремих блоків.

## Висновки

1. Блокова модель визначає розміри кожного елемента на веб-сторінці. Використання `box-sizing: border-box` є необхідним стандартом для передбачуваного розмічання.
2. Тип відображення (`display`) визначає, чи буде елемент переносити рядок і чи можна йому вручну задавати розміри. Блокові, рядкові та рядково-блокові мають принципові відмінності у поведінці.
3. Для зміни візуального розташування елементів без зміни HTML-структури використовується властивість `position`. Найпопулярніша комбінація — `position: relative` для батька та `position: absolute` для нащадка.
4. Властивість `z-index` керує шарами перекриття, але вимагає обережності через формування локальних контекстів накладання. Може викликати складні для відладки баги.

## Джерела


1. MDN Web Docs: CSS Box Model
2. MDN Web Docs: Position
3. CSS-Tricks: The z-index Property and Stacking Contexts

4. Learn CSS (web.dev) - Box Model

## Запитання для самоперевірки

1. Які чотири складові містить блокова модель CSS?
2. Що змінює правило `box-sizing: border-box` у розрахунку розмірів елемента?
3. В чому різниця між властивостями `display: none` та `visibility: hidden`?
4. Які стилі застосовуватимуться до елемента, якщо він є `display: inline` і йому задати `width: 100px` та `margin-top: 50px`?
5. Що таке "схлопування зовнішніх відступів" (margin collapsing)?
6. Відносно чого позиціюється елемент із `position: absolute`? Що станеться, якщо у нього немає жодного спозиційованого батьківського елемента?
7. Чим `position: sticky` відрізняється від `position: fixed`?
8. Чому елемент із `z-index: 999` може знаходитись ПІД елементом із `z-index: 2`? У яких випадках це відбувається?